Laboratório Funcional · Aula 12

Matheus Eduardo Polizelli · RA 24129411-2 · Turma A
Gerado em 16/10/2026, 21:05:38 · tempo registrado no cronômetro: 0:00

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

def registra(msg): print(msg)
é impura pois possui efeito colateral (imprimir no console)

def sorteia(): return random.randint(1, 6)
é impura pois diferentes execuções com o mesmo parâmetro não necessariamente produzem o mesmo resultado

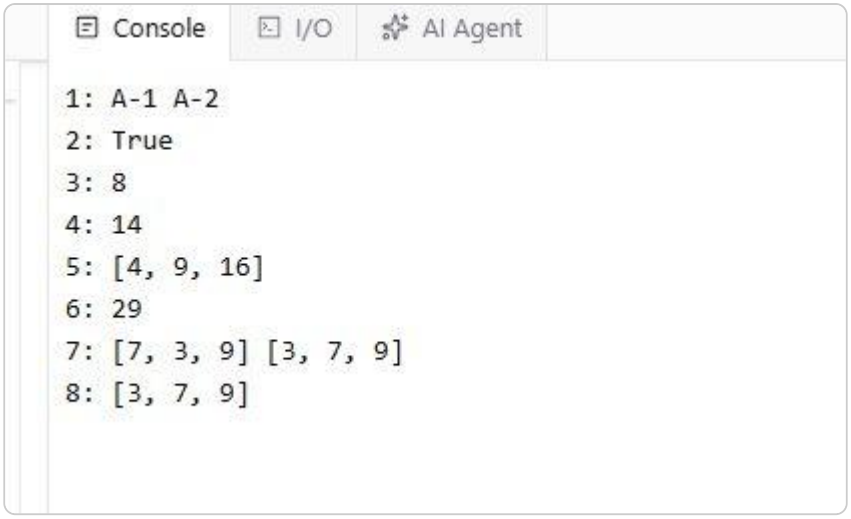
Prática 1: Funções puras e formas funcionais

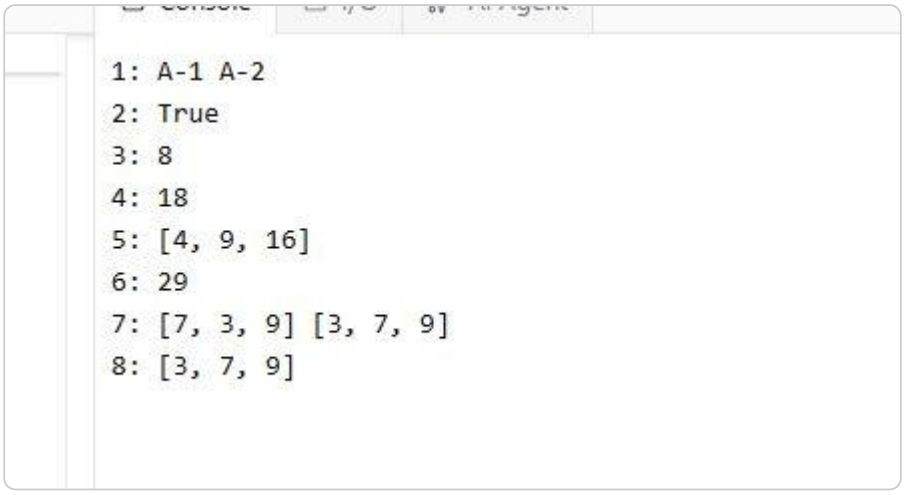
Previsão: O que as linhas 1, 4 e 7 vão imprimir?

A-1 A-2
14
[7, 3, 9] [3, 7, 9]

Saídas (original e com compor(g, f))

não respondido





1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

Pelo contador ser uma variável global, diferentes execuções irão trazer diferentes resultados. Isso dificulta os testes pois nem toda a execução traz o mesmo resultado e por isso é difícil fazer uma asserção precisa

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

Os resultados são diferentes porque a ordem de aplicação das funções muda. Na primeira composição, multiplicamos por 3 e depois somamos 2. Na segunda, somamos 2 e depois multiplicamos por 3. Portanto, a ordem das funções altera o resultado.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

o sorted(notas) cria uma lista nova, o notas.sort() modifica a existente. sorted(notas) segue o estilo funcional pois não está alterando uma variável externa.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

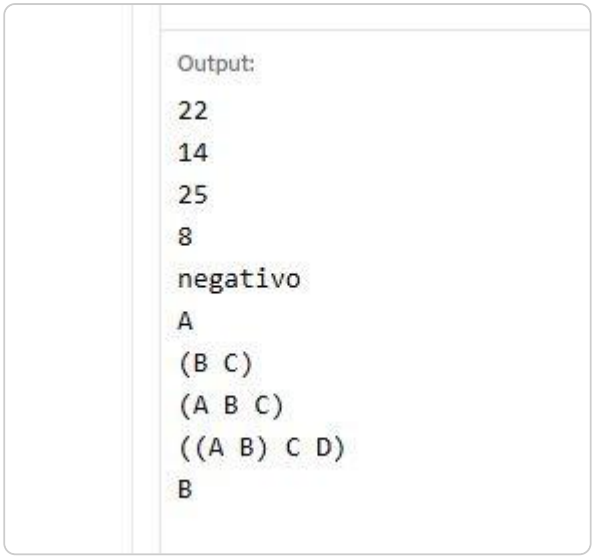
B C

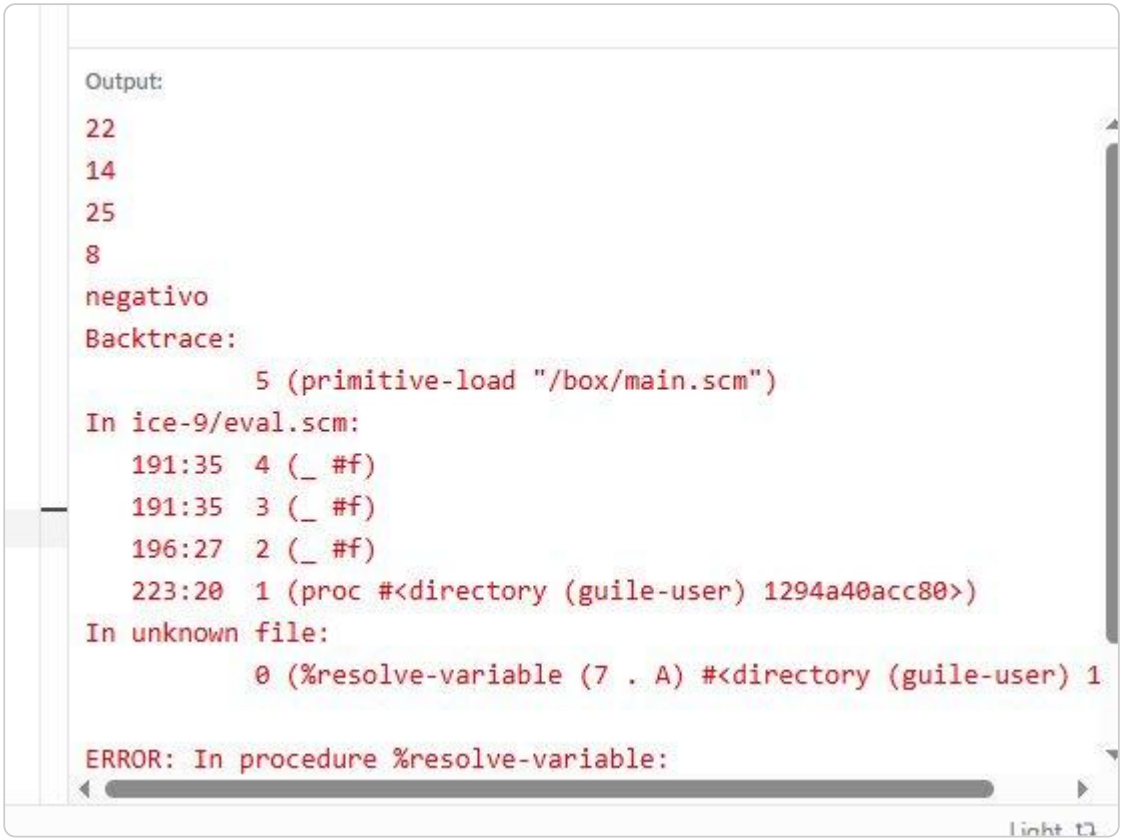
Previsão: (cons '(A B) '(C D))

A B C D

Saídas (original e sem o apóstrofo)

não respondido





2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo (') no Scheme serve para impedir que o Scheme tente executar ou avaliar uma expressão, tratando-a como um dado.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(car (cdr (cdr '(A B C))))
Resultado: C
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

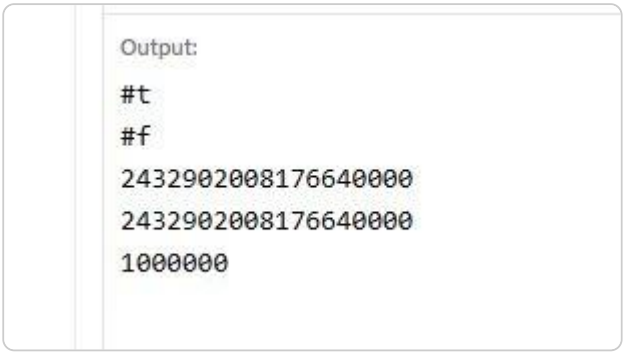
n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).
n = 6: comum em 12 passos, até 5 multiplicações esperando (expressão de até 42 caracteres); em cauda em 8 passos, nada esperando (até 18 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

Provavelmente os dois não irão funcionar

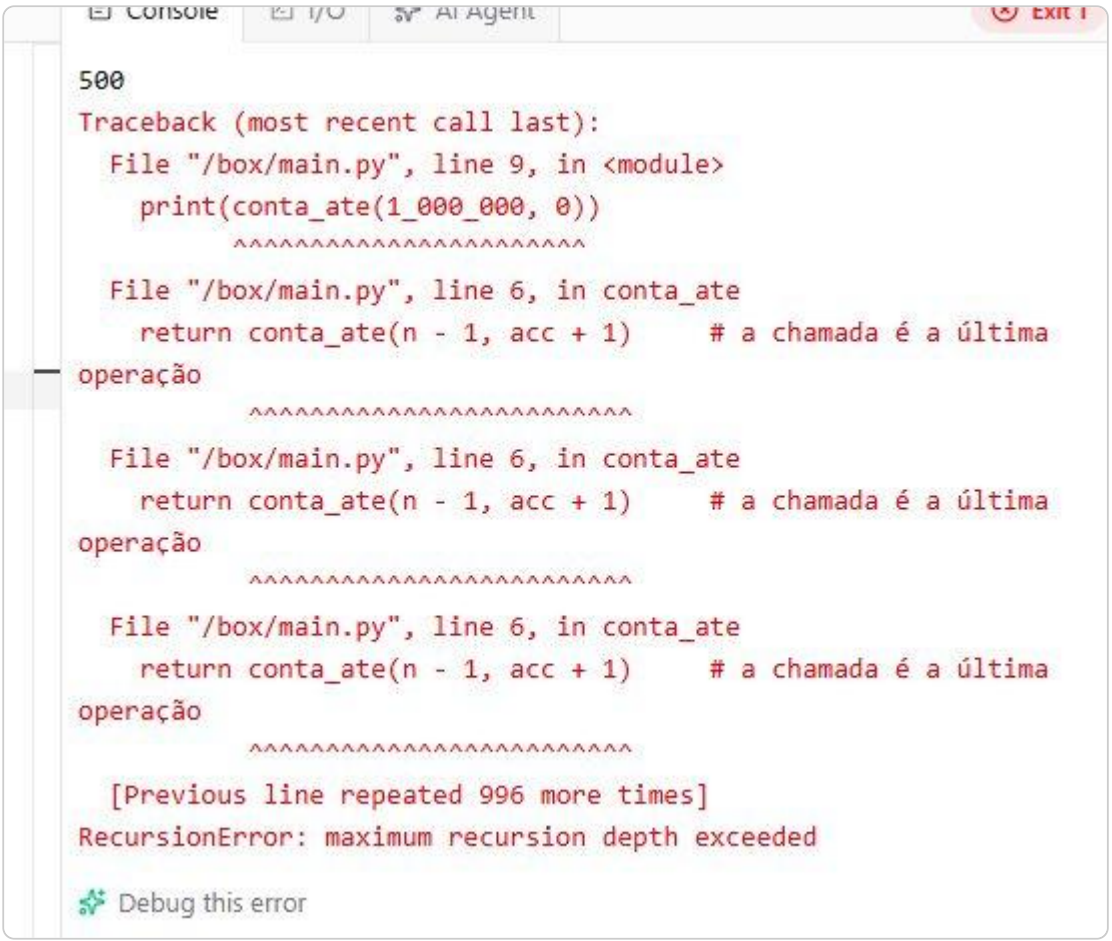
Saída do Scheme

não respondido



Saída do Python (as últimas linhas bastam)

não respondido



3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum, cresce a pilha de chamadas, porque cada chamada precisa esperar a próxima terminar para continuar o cálculo. Na recursão em cauda, a pilha não precisa crescer da mesma forma, porque a chamada recursiva é a última operação da função

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme conseguiu fazer um milhão de chamadas porque otimiza a chamada em cauda, reutilizando o espaço da pilha em vez de guardar uma nova chamada a cada repetição. No Python, cada chamada fica registrada na pilha, até atingir o limite de recursão e gerar o erro RecursionError.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

não respondido

```
Output:
tamanho: 0  (esperado 3)
soma-quadrados: 0  (esperado 14)
pares: ()  (esperado (2 4 6))
```

```
Output:
tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis))
         (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis))
         (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

tamanho: usei null? para verificar se a lista estava vazia e cdr para percorrer o restante da lista. Somei 1 para cada elemento.

soma-quadrados: usei null? para identificar o fim da lista, car para pegar o primeiro elemento e calcular seu quadrado, e cdr para continuar com os outros elementos.

pares: usei null? para verificar o fim da lista, car para pegar o primeiro elemento, even? para verificar se ele era par e cdr para continuar percorrendo a lista. Usei cons para montar a lista com os números pares.

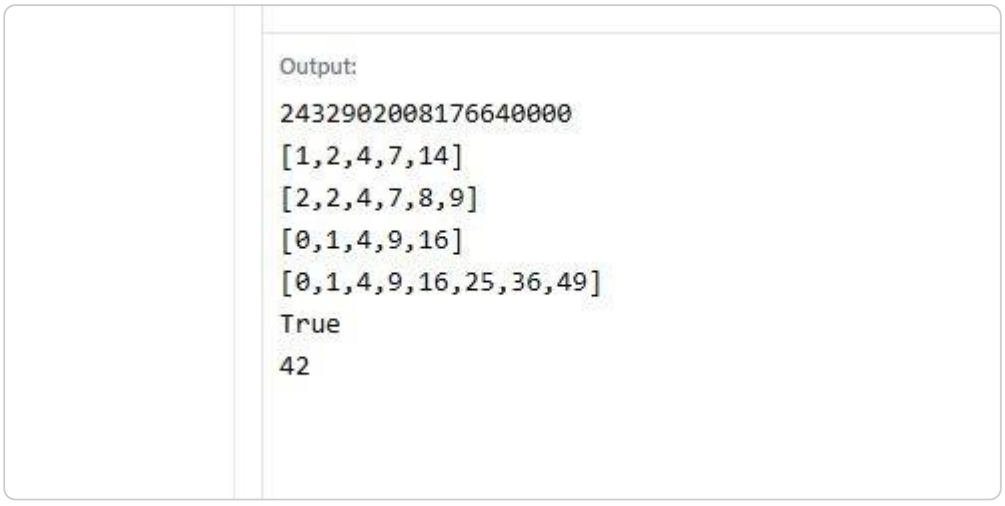
Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

1, 4, 9, 16, 25
42

Saídas (original e com member 17)

não respondido





5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell consegue imprimir parte de uma lista infinita por causa da avaliação preguiçosa. Ele calcula os elementos somente quando são necessários
No caso de 42 (div 1 0), a função primeiro x _ = x retorna apenas o primeiro argumento (42) e não avalia o segundo argumento

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

O member 17 squares nunca termina porque a lista squares contém os quadrados perfeitos: 0, 1, 4, 9, 16, 25, 36.... Como 17 não é um quadrado perfeito, a função continua procurando em uma lista infinita, sem encontrar o valor.

Já member 16 squares termina porque encontra o número 16 na lista e retorna True.
Como a lista está em ordem crescente, podemos parar a busca quando o elemento atual for maior que o valor procurado

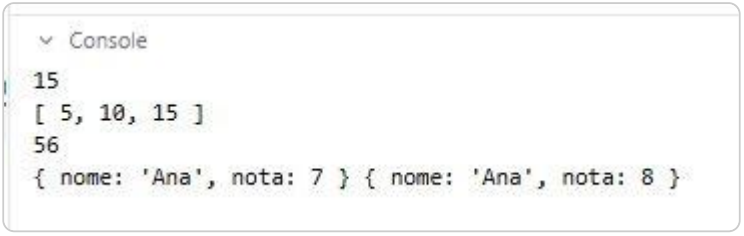
Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

15
[5, 10, 15]

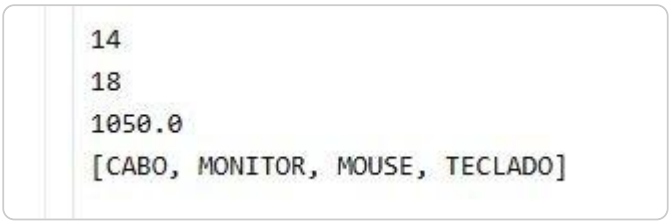
Saída do JavaScript

não respondido



Saída do Java

não respondido



6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

Quando somador(10) é executada, ela devolve uma função interna (x) => x + n. Essa função mantém acesso à variável n, que vale 10, mesmo depois que a execução de somador termina. Isso ocorre através do fechamento (closure)

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

Os resultados são diferentes porque compose e andThen aplicam as funções em ordens diferentes. f.compose(g) executa primeiro g(4) = 12 e depois f(12) = 14. Já f.andThen(g) executa primeiro f(4) = 6 e depois g(6) = 18.
O método que corresponde à função compor(f, g) da Prática 1 é o compose, pois calcula f(g(x)) aplicando primeiro g e depois f

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
```

```
];

const aprovados = alunos
  .filter(aluno ⇒ aluno.nota ⩾ 6)
  .map(aluno ⇒ aluno.nome.toUpperCase());

console.log(aprovados.join(", "));
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
precos = [50.0, 120.0, 80.0, 200.0]

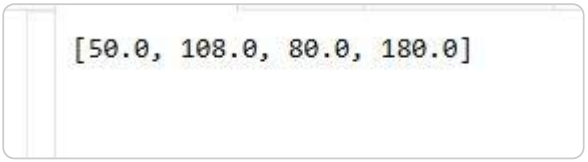
com_desconto = [p * 0.9 if p > 100 else p for p in precos]

print(com_desconto)
```

Saídas das suas versões

não respondido





7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

Na minha visão a funcional se tornou mais fácil para ler, porém a depuração é melhor com o laço

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A funcional é em geral mais fácil de ler pela simplicidade e expressividade. Em sistemas normais é plenamente aceitável, porém em sistemas em que toda otimização voltada ao desempenho é necessário, não aceitaria esse trade-off

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

A recursão infinita, não sabia que poderia existir.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

não respondido